Patient Health Monitoring Data Pipeline using AWS

BUSINESS SCENARIO
A hospital receives patient health monitoring files every hour from:
- ICU monitoring systems
- Wearable health devices
- Hospital management systems
- Emergency care systems

The hospital wants to:
- Store incoming patient data securely
- Validate health records automatically
- Detect corrupted files
- Clean and transform patient data
- Catalog metadata automatically
- Store processed healthcare data
- Separate bad patient records
- Maintain production-ready ETL workflows

Objective
Build a healthcare ETL pipeline that processes patient monitoring data securely.

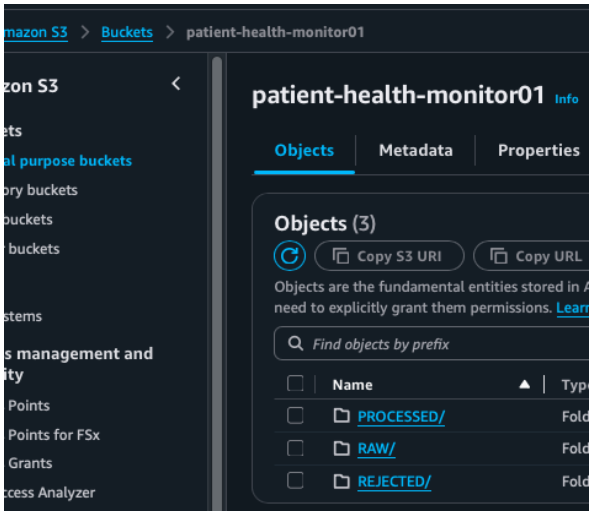
PROJECT GOALS
build a system that:
- Ingests patient data
- Validates incoming files
- Cleans medical records
- Detects invalid patient readings
- Stores transformed healthcare data
- Handles schema changes

HEALTH METRICS TO TRACK
Metric	Example
Heart Rate	72 BPM
Blood Pressure	120/80

Oxygen Level	98%
Temperature	98.6 F
Respiration Rate	16/min

S3 storage – patient s-health-monitor-bucket

raw/
  Vitals/
  Patient/
  Device/
Processed/
  Vitals/
  Patients/
  Reports /
Rejected/



CREATION OF IAM IMPLEMENTATION

Glue IAM Role
Lambda IAM Role

SAMPLE HEALTHCARE DATA

FILE 1 — patient_vitals.csv

patient_id,timestamp,heart_rate,blood_pressure,oxygen_level,temperature
P1001,2025-01-10 08:00:00,72,120/80,98,98.6
P1002,2025-01-10 08:05:00,95,140/90,96,99.1
P1003,2025-01-10 08:10:00,45,90/60,88,101.2
P1004,2025-01-10 08:15:00,,130/85,97,98.4
P1005,2025-01-10 08:20:00,110,150/95,,100.1
P1005,2025-01-10 08:20:00,110,150/95,,100.1

FILE 2 — patient_details.csv

patient_id,name,age,gender,city,disease
P1001,Rahul Sharma,45,Male,Delhi,Diabetes
P1002,Priya Verma,52,Female,Mumbai,Hypertension
P1003,Amit Singh,60,Male,Pune,Heart Disease
P1004,Neha Kapoor,29,Female,Delhi,Asthma
P1005,Rohit Jain,48,Male,Jaipur,Diabetes

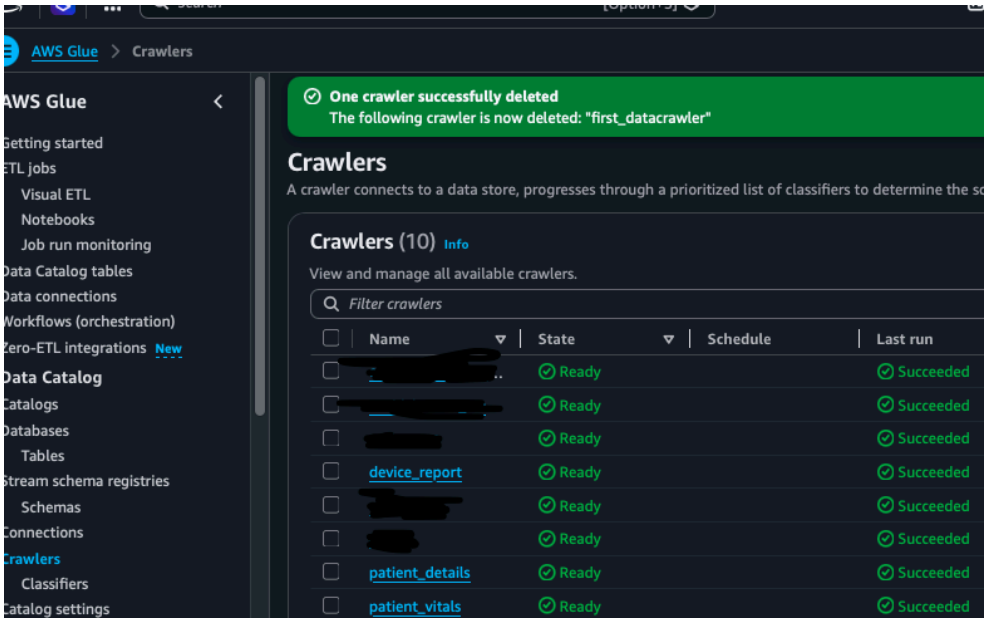
FILE 3 — device_logs.csv

device_id,patient_id,device_type,status,last_sync
D101,P1001,SmartWatch,Active,2025-01-10 08:00:00
D102,P1002,Oximeter,Inactive,2025-01-10 07:50:00
D103,P1003,HeartMonitor,Active,2025-01-10 08:05:00

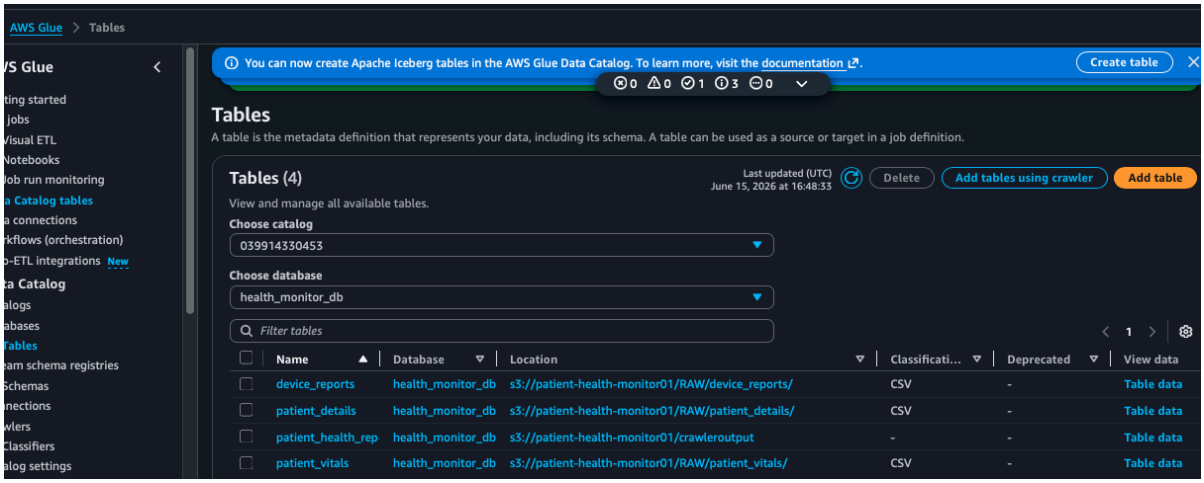
DETECT FOLLOWING:

Issue	Example
Null values	heart_rate missing
Duplicate records	P1005 repeated
Critical health readings	oxygen_level = 88
Missing patient metrics	oxygen_level blank
Schema changes	new column added

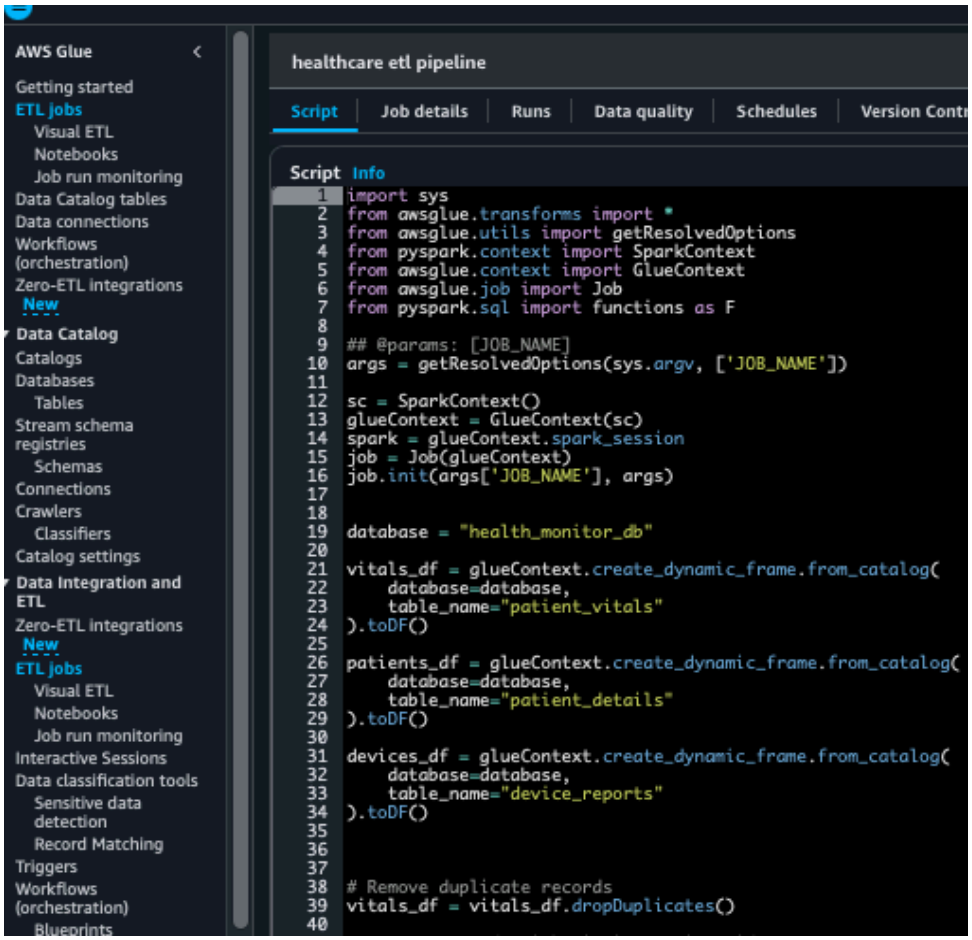
AWS CRAWLER TO READ HEALTH CARE BUCKET RAW DATA
Automatically discover healthcare schema.

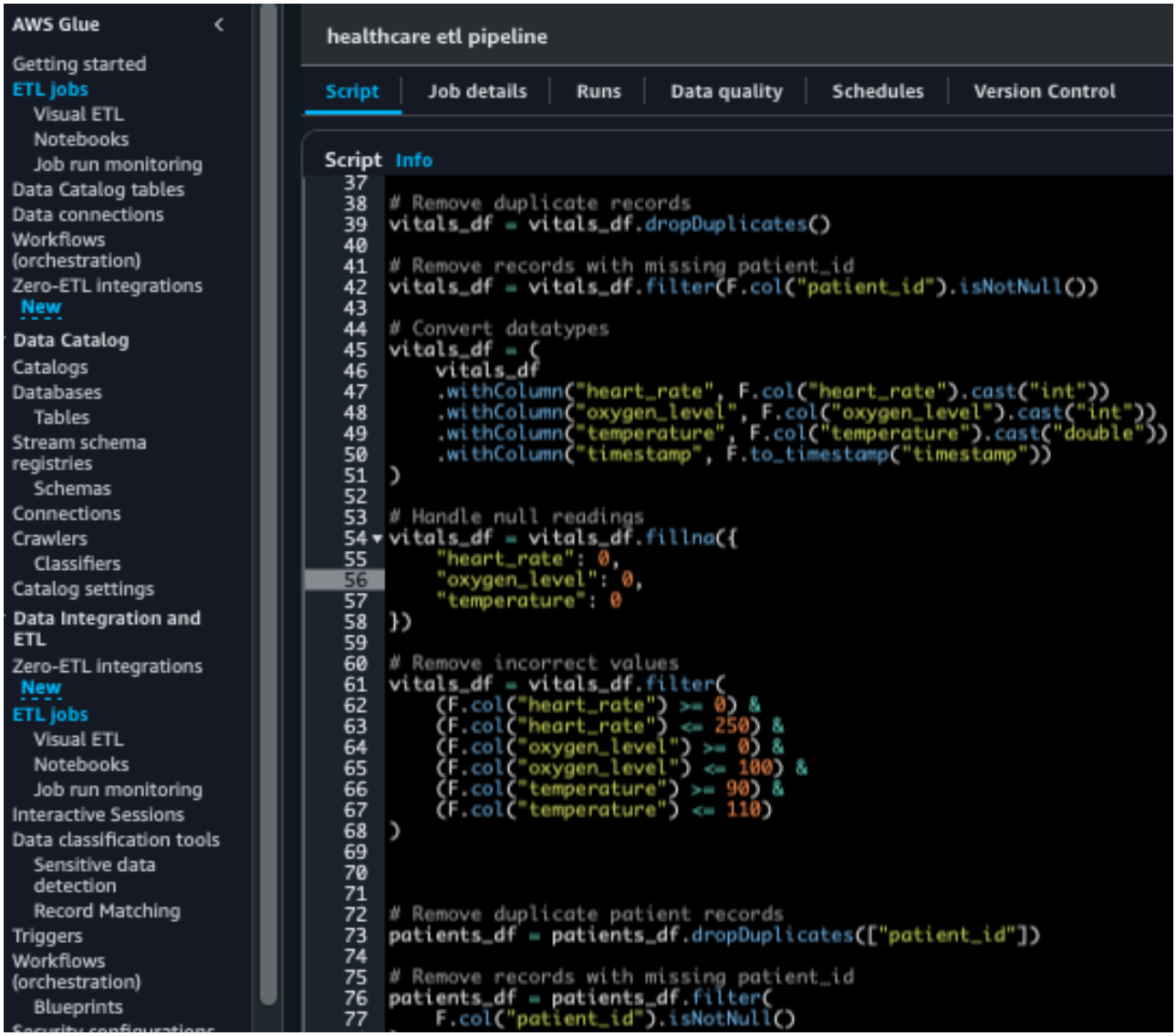


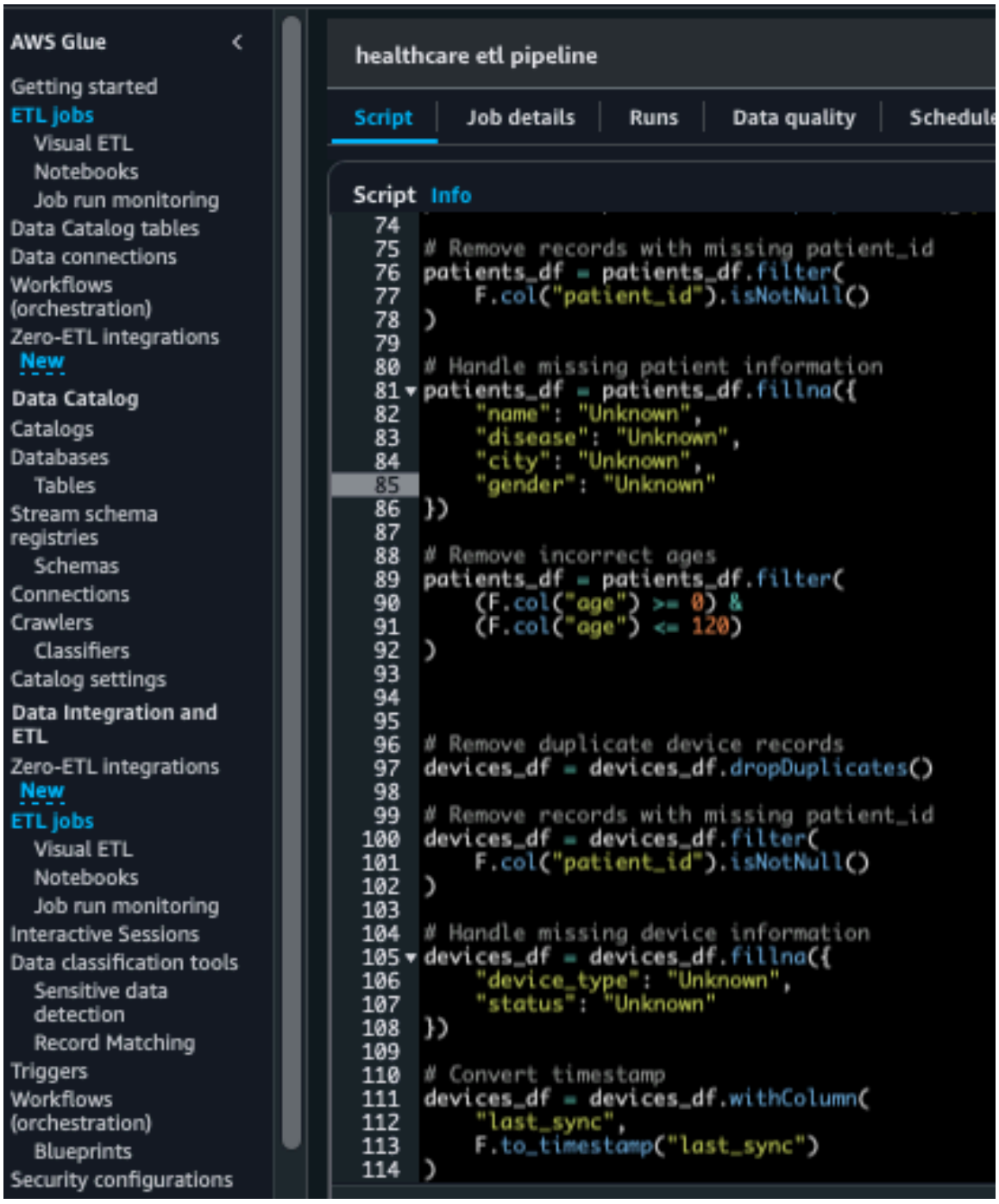
DATABASE AND TABLE AUTOMATICALLY SCANNED FROM RAW BUCKET

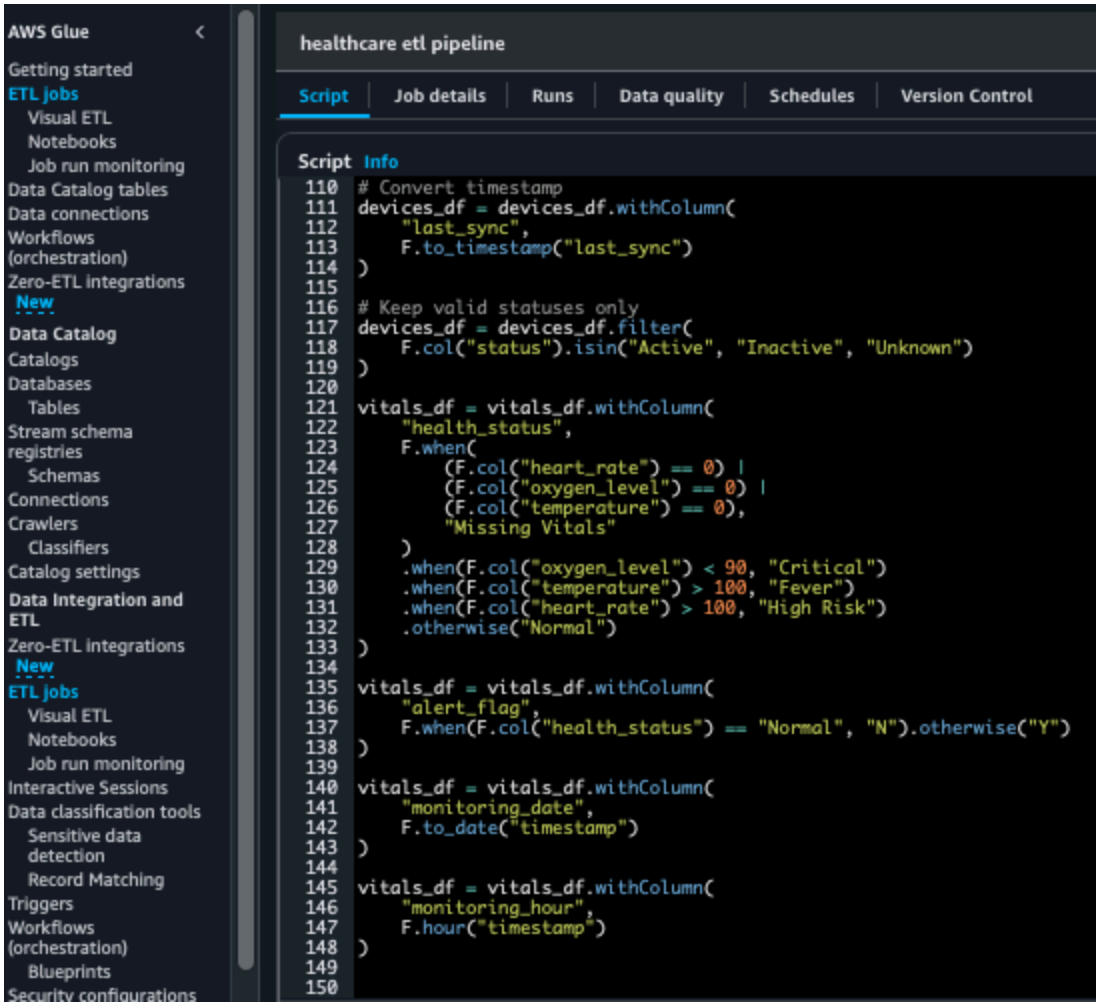


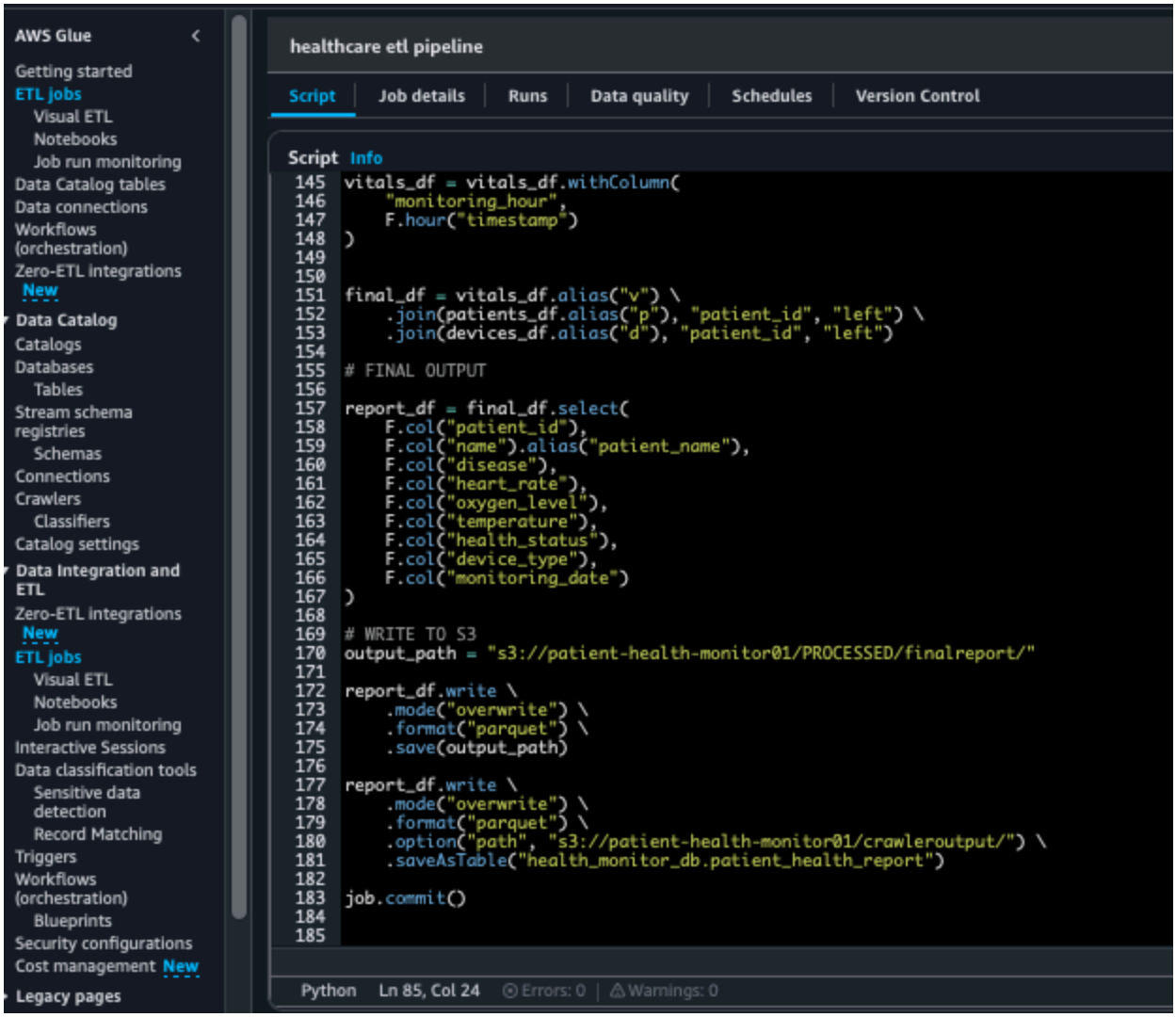
MAIN HEALTHCARE ETL PIPELINE USING PYSPARK



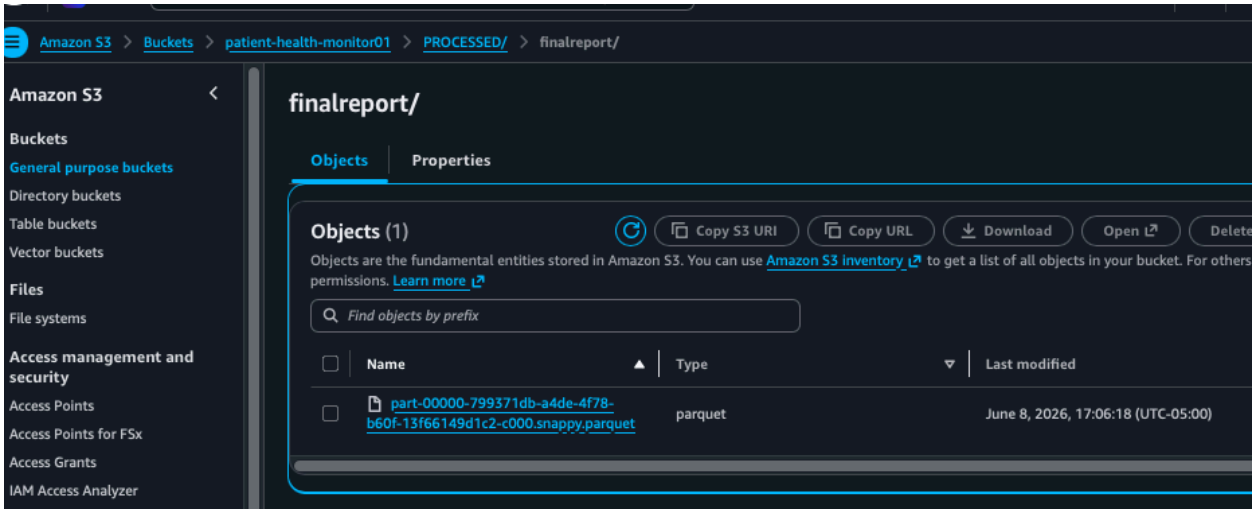

AWS Glue
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations
New
Data Catalog
Catalogs
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
Data Integration and ETL
Zero-ETL integrations
New
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Interactive Sessions
Data classification tools
Sensitive data detection
Record Matching
Triggers
Workflows (orchestration)
Blueprints
healthcare etl pipeline
Script
Job details
Runs
Data quality
Schedules
Version Control
Script Info
37
38 # Remove duplicate records
39 vitals_df = vitals_df.dropDuplicates()
40
41 # Remove records with missing patient_id
42 vitals_df = vitals_df.filter(F.col("patient_id").isNotNull())
43
44 # Convert datatypes
45 vitals_df = (
46 vitals_df
47 .withColumn("heart_rate", F.col("heart_rate").cast("int"))
48 .withColumn("oxygen_level", F.col("oxygen_level").cast("int"))
49 .withColumn("temperature", F.col("temperature").cast("double"))
50 .withColumn("timestamp", F.to_timestamp("timestamp"))
51)
52
53 # Handle null readings
54 vitals_df = vitals_df.fillna({
55 "heart_rate": 0,
56 "oxygen_level": 0,
57 "temperature": 0
58 })
59
60 # Remove incorrect values
61 vitals_df = vitals_df.filter(
62 (F.col("heart_rate") >= 0) &
63 (F.col("heart_rate") <= 250) &
64 (F.col("oxygen_level") >= 0) &
65 (F.col("oxygen_level") <= 100) &
66 (F.col("temperature") >= 90) &
67 (F.col("temperature") <= 110)
68)
69
70
71
72 # Remove duplicate patient records
73 patients_df = patients_df.dropDuplicates(["patient_id"])
74
75 # Remove records with missing patient_id
76 patients_df = patients_df.filter(
77 F.col("patient_id").isNotNull()


AWS Glue
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations
New
Data Catalog
Catalogs
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
Data Integration and ETL
Zero-ETL integrations
New
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Interactive Sessions
Data classification tools
Sensitive data detection
Record Matching
Triggers
Workflows (orchestration)
Blueprints
Security configurations
healthcare etl pipeline
Script
Job details
Runs
Data quality
Schedule
Script Info
74
75 # Remove records with missing patient_id
76 patients_df = patients_df.filter(
77 F.col("patient_id").isNotNull()
78)
79
80 # Handle missing patient information
81 patients_df = patients_df.fillna({
82 "name": "Unknown",
83 "disease": "Unknown",
84 "city": "Unknown",
85 "gender": "Unknown"
86 })
87
88 # Remove incorrect ages
89 patients_df = patients_df.filter(
90 (F.col("age") >= 0) &
91 (F.col("age") <= 120)
92)
93
94
95
96 # Remove duplicate device records
97 devices_df = devices_df.dropDuplicates()
98
99 # Remove records with missing patient_id
100 devices_df = devices_df.filter(
101 F.col("patient_id").isNotNull()
102)
103
104 # Handle missing device information
105 devices_df = devices_df.fillna({
106 "device_type": "Unknown",
107 "status": "Unknown"
108 })
109
110 # Convert timestamp
111 devices_df = devices_df.withColumn(
112 "last_sync",
113 F.to_timestamp("last_sync")
114)

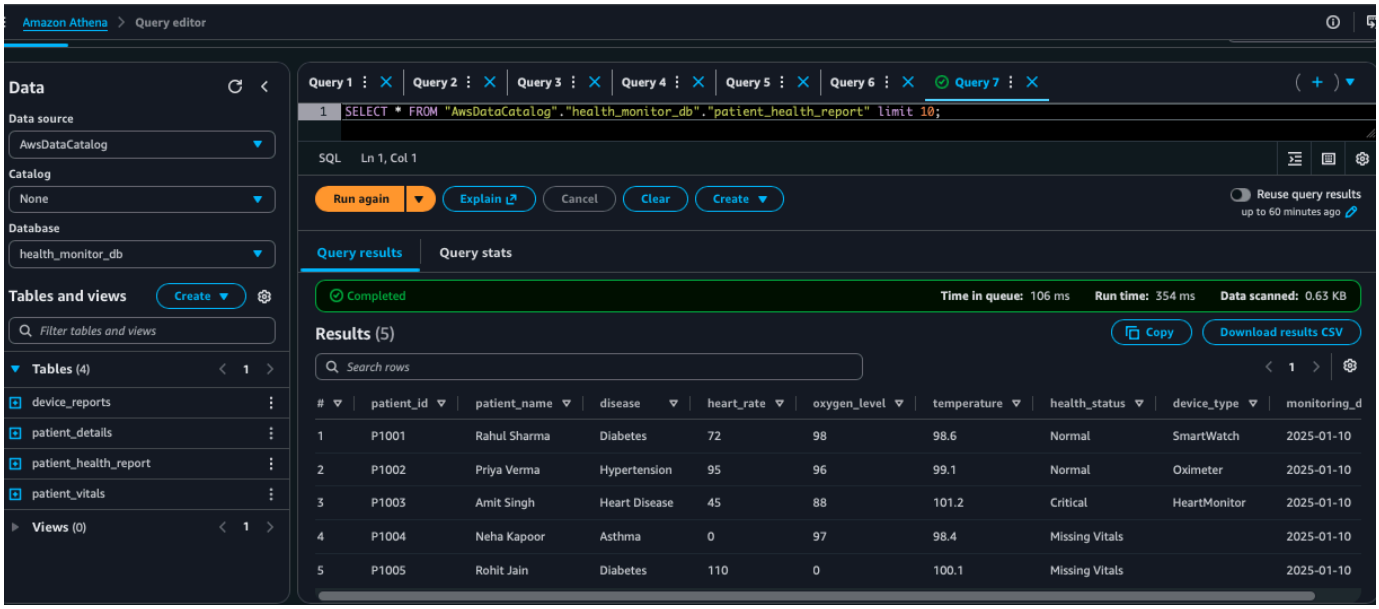

AWS Glue
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations
New
Data Catalog
Catalogs
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
Data Integration and ETL
Zero-ETL integrations
New
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Interactive Sessions
Data classification tools
Sensitive data detection
Record Matching
Triggers
Workflows (orchestration)
Blueprints
Security configurations
healthcare etl pipeline
Script
Job details
Runs
Data quality
Schedules
Version Control
Script Info
# Convert timestamp
devices_df = devices_df.withColumn(
"last_sync",
F.to_timestamp("last_sync")
)
# Keep valid statuses only
devices_df = devices_df.filter(
F.col("status").isin("Active", "Inactive", "Unknown")
)
vitals_df = vitals_df.withColumn(
"health_status",
F.when(
(F.col("heart_rate") == 0) |
(F.col("oxygen_level") == 0) |
(F.col("temperature") == 0),
"Missing Vitals"
)
.when(F.col("oxygen_level") < 90, "Critical")
.when(F.col("temperature") > 100, "Fever")
.when(F.col("heart_rate") > 100, "High Risk")
.otherwise("Normal")
)
vitals_df = vitals_df.withColumn(
"alert_flag",
F.when(F.col("health_status") == "Normal", "N").otherwise("Y")
)
vitals_df = vitals_df.withColumn(
"monitoring_date",
F.to_date("timestamp")
)
vitals_df = vitals_df.withColumn(
"monitoring_hour",
F.hour("timestamp")
)



PROCESSED DATA IN PARQUET FORMAT



OUTPUT


Amazon Athena
Query editor
Data
Data source
AwsDataCatalog
Catalog
None
Database
health_monitor_db
Tables and views
Create
Filter tables and views
Tables (4)
device_reports
patient_details
patient_health_report
patient_vitals
Views (0)
Query 1
Query 2
Query 3
Query 4
Query 5
Query 6
Query 7
SELECT * FROM "AwsDataCatalog"."health_monitor_db"."patient_health_report" limit 10;
SQL
Ln 1, Col 1
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 106 ms
Run time: 354 ms
Data scanned: 0.63 KB
Results (5)
Copy
Download results CSV
Search rows
patient_id patient_name disease heart_rate oxygen_level temperature health_status device_type monitoring_d
1 P1001 Rahul Sharma Diabetes 72 98 98.6 Normal SmartWatch 2025-01-10
2 P1002 Priya Verma Hypertension 95 96 99.1 Normal Oximeter 2025-01-10
3 P1003 Amit Singh Heart Disease 45 88 101.2 Critical HeartMonitor 2025-01-10
4 P1004 Neha Kapoor Asthma 0 97 98.4 Missing Vitals 2025-01-10
5 P1005 Rohit Jain Diabetes 110 0 100.1 Missing Vitals 2025-01-10